

Build Automation Tools — Gradle

Ingeniería de Sistemas

Universidad Austral — 2026

El problema

Cuando un proyecto crece en tamaño, personas, duración o cantidad de *outputs*, se vuelve complejo de manejar.

Hay tareas que se repiten constantemente: compilar, testear, generar artefactos, etc.

- Las ejecutan distintas personas, en distintos entornos, a lo largo del tiempo
- Algunas tareas dependen de otras → hay un **orden obligatorio**
- Hacerlo a mano genera inconsistencias

El problema — compilar una app Java

Sin herramienta, a mano:

```
# 1. descargar dependencias manualmente
curl -O https://repo1.maven.org/maven2/.../httpclient-4.5.5.jar

# 2. compilar pasando el classpath a mano
javac -cp lib/httpclient.jar:lib/gson.jar -d out src/**/*.java

# 3. empaquetar
jar cfm app.jar MANIFEST.MF -C out .

# 4. repetir en cada máquina, en cada entorno
```

El problema — armar el *bundle* de una web

Sin herramienta, a mano:

```
# 1. transpilar TypeScript
npx tsc --outDir dist/

# 2. resolver imports y empaquetar
npx esbuild dist/index.js --bundle --outfile=bundle.js

# 3. minificar
npx terser bundle.js -o bundle.min.js

# 4. copiar assets
cp -r src/assets/ dist/assets/
```

¿Qué es una *build automation tool*?

Herramienta que permite **automatizar y estandarizar** los procesos del desarrollo de software.

Su propósito más básico: **ejecutar tareas**.

- Modelan una **red de dependencias** entre tareas
- Al ejecutar un objetivo, calculan el orden correcto automáticamente
- Cada tarea dependiente se ejecuta **exactamente una vez**

Ejemplos

Ecosistema	Herramienta
C/C++	Make, CMake
.NET	NAnt, MSBuild
Java	Ant, Maven, Gradle
Ruby	Rake, Buildr
.NET / PowerShell	Psake

Gradle

Gradle — características

Gradle es una herramienta de *build open-source*, diseñada para ser lo suficientemente flexible como para construir casi cualquier tipo de *software*.

- **Propósito general** — no limitado a un lenguaje
- **Base JVM** — corre en la JVM
- **Modelo de tareas** — todo gira alrededor de *tasks*
- **Multi-proyecto** — soporta builds con múltiples módulos
- **Soporte de IDE** — IntelliJ, Eclipse, VS Code
- **Resolver de dependencias** — integrado con Maven Central, etc.

Workshop

Ejemplo básico

Inicializar un proyecto

```
gradle init
```

Para este ejemplo usamos una configuración "básica" (*basic*).

Gradle vs **Gradlew**

Gradle genera un **Gradle Wrapper**, que fija la versión de Gradle para un entorno específico.

```
./gradlew tasks --all # usar siempre el wrapper
```

Crear una *task*

En `build.gradle`:

```
tasks.register('hello') {  
    doLast {  
        println 'hello'  
    }  
}
```

```
./gradlew tasks --all    # la task aparece listada  
./gradlew hello          # ejecutar
```

Dependencias entre *tasks*

En `build.gradle` :

```
tasks.register('taskX') {  
    dependsOn 'taskY'  
    doLast { println 'taskX' }  
}  
  
tasks.register('taskY') {  
    doLast { println 'taskY' }  
}
```

Al ejecutar `./gradlew taskX`, Gradle corre primero `taskY`.

Configuraciones

¿Qué es una *configuration*?

"A Configuration represents a group of artifacts and their dependencies."

Se usan para agrupar dependencias según su propósito (compilar, testear, ejecutar, etc.).

```
configurations {  
    inggis  
}
```

```
./gradlew resolvableConfigurations --all
```

Agregar dependencias a una *configuration*

En `build.gradle`:

```
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    inggis 'org.apache.httpcomponents:httpclient:4.5.5'  
}
```

```
./gradlew dependencies
```

Task que depende de una configuration

```
tasks.register('listIngsis') {  
    dependsOn configurations.ingsis  
    doLast {  
        def files = configurations.ingsis.collect { File f -> f.name }  
        println "dependencies = ${files}"  
    }  
}
```

```
./gradlew listIngsis
```

Plugins

¿Qué es un *plugin*?

"Gradle at its core intentionally provides very little for real world automation. All of the useful features, like the ability to compile Java code, are added by plugins."

Un *plugin* agrega:

- Nuevas **tasks** (ej: `compileJava`, `test`, `jar`)
- **Convenciones** (estructura de directorios)
- Nuevos **objetos de dominio**

Agregar el plugin java

En `build.gradle`:

```
plugins {  
    id 'java'  
}
```

```
./gradlew tasks           # aparecen tasks de Java  
./gradlew dependencies    # aparecen configurations de Java  
./gradlew compileJava     # compilar (sin código aún, no hace nada)
```

Agregar código Java

Crear la estructura:

```
src/main/java/edu/austral/ingsis/gradle/App.java
```

```
public class App {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

```
./gradlew compileJava    # ahora sí compila
```

Plugin application

```
plugins {  
    id 'java'  
    id 'application'  
}  
  
application {  
    mainClass = "edu.austral.ingsis.gradle.App"  
}
```

```
./gradlew tasks    # aparece la task 'run'  
./gradlew run      # ejecuta la aplicación  
./gradlew clean    # limpia el directorio build/
```

Multi-proyecto

Inicializar un multi-proyecto

Limpiar y reinicializar:

```
rm -rf build build.gradle gradle gradlew gradlew.bat settings.gradle src
gradle init
# Elegir: Application / Java / Multi-project
```

```
./gradlew projects      # listar sub-proyectos
./gradlew tasks         # tasks del root
./gradlew tasks --all   # todas las tasks
./gradlew run           # ejecutar desde el root
./gradlew :app:run      # ejecutar solo el sub-proyecto 'app'
```

Conceptos a explorar en multi-proyecto

- **Dependencias entre módulos** — agregar y quitar una dependencia entre sub-proyectos
- **Sub-tasks** — ejecutar tareas específicas de un módulo
- **Orden de tasks** — ver el orden con `--dry-run`
- **Configuraciones comunes** — definir config compartida en el proyecto raíz
- **Convenciones** — agregar y quitar convenciones globales
- **Task** `test` — configurar el comportamiento de tests en todos los módulos

Referencia

Repositorio de ejemplo: <https://github.com/austral-ingsis/gradle-basic>

Documentación oficial: <https://docs.gradle.org/current/userguide/userguide.html>